



/**

Contexto:

Você foi contratado para desenvolver um sistema de gerenciamento de tarefas (To-Do List) onde cada tarefa possui uma prioridade e pode ser processada de diferentes formas. O sistema deve usar lista encadeada para armazenar as tarefas e ponteiros para função para implementar diferentes estratégias de processamento.

Implemente o seguinte:

*/

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

// Estrutura para representar uma tarefa

typedef struct tarefa {

int id;

char descricao[100];

int prioridade; // 1-Baixa, 2-Média, 3-Alta

struct tarefa *prox;

} Tarefa;

// Ponteiro para função de processamento

typedef void (*FuncaoProcessamento)(Tarefa*);

Tarefa* criar_tarefa(int id, const char *descricao, int prioridade) {

/**

* COMPLETE ESTA FUNÇÃO (0,5 pontos)

* Aloca e inicializa uma nova tarefa

* Retorna ponteiro para a tarefa criada

*/

}

void inserir_tarefa(Tarefa **lista, int id, const char *descricao, int prioridade) {

/**

* COMPLETE ESTA FUNÇÃO (0,5 pontos)

* Insere nova tarefa no FINAL da lista

* Mantenha o ponteiro para o último elemento para eficiência

*/

}

// Funções de processamento (JÁ IMPLEMENTADAS)

void imprimir_detalhado(Tarefa *t) {

printf("Tarefa %d: %s [Prioridade: %d]\n", t->id, t->descricao, t->prioridade);

}

void imprimir_resumido(Tarefa *t) {

printf("%d - %s\n", t->id, t->descricao);

}

void processar_alta_prioridade(Tarefa *t) {

if (t->prioridade == 3) {

printf("URGENTE: %s\n", t->descricao);

}

}

void processar_lista(Tarefa *lista, FuncaoProcessamento funcao) {

/**

* COMPLETE ESTA FUNÇÃO (0,5 pontos)

* Percorre a lista e aplica a função de processamento em cada tarefa

* Use ponteiro para função conforme o parâmetro 'funcao'

*/

}

void liberar_lista(Tarefa *lista) {

/**

* COMPLETE ESTA FUNÇÃO (0,5 pontos)

* Libera toda a memória alocada para a lista

*/

}

int main() {

Tarefa *lista = NULL;

// Criando algumas tarefas

inserir_tarefa(&lista, 1, "Estudar para a prova", 3);

inserir_tarefa(&lista, 2, "Fazer compras", 2);

inserir_tarefa(&lista, 3, "Reunião importante", 3);

inserir_tarefa(&lista, 4, "Ler livro", 1);

printf("=== LISTA DETALHADA ===\n");

processar_lista(lista, imprimir_detalhado);

printf("\n=== LISTA RESUMIDA ===\n");

processar_lista(lista, imprimir_resumido);

printf("\n=== TAREFAS URGENTES ===\n");

processar_lista(lista, processar_alta_prioridade);

liberar_lista(lista);

return 0;

}

/**

Saída esperada:

=== LISTA DETALHADA ===

Tarefa 1: Estudar para a prova [Prioridade: 3]

Tarefa 2: Fazer compras [Prioridade: 2]

Tarefa 3: Reunião importante [Prioridade: 3]

Tarefa 4: Ler livro [Prioridade: 1]

=== LISTA RESUMIDA ===

1 - Estudar para a prova

2 - Fazer compras

3 - Reunião importante

4 - Ler livro

=== TAREFAS URGENTES ===

URGENTE: Estudar para a prova

URGENTE: Reunião importante

=====

CRITÉRIOS DE AVALIAÇÃO:

criar_tarefa (1,0 ponto):

Alocação correta de memória (0,4)

Inicialização dos campos (0,4)

Retorno do ponteiro (0,2)

inserir_tarefa (1,0 ponto):

Inserção no final da lista (0,4)

Tratamento de lista vazia (0,4)

Manutenção do ponteiro correto (0,2)

processar_lista (1,0 ponto):

Percorre toda a lista (0,4)

Uso correto do ponteiro para função (0,4)

Chamada adequada da função (0,2)

liberar_lista (1,0 ponto):

Libera todos os nós (0,6)

Não deixa memory leaks (0,4)

Criar uma função que permita o usuário inserir

suas tarefas (0,5 ponto)

Criar uma função que permita o usuário deletar

suas tarefas (0,5 ponto)

=====

/**

Título:

Manipulação de Lista Encadeada com Operações Alternadas (Dobrar / Somar 10)

Objetivo:

Desenvolver um programa modular em linguagem C que utilize listas encadeadas para armazenar números inteiros informados pelo usuário. O programa deve aplicar operações distintas em nós de posições pares e ímpares, utilizando ponteiros para função (callbacks) para realizar essas operações de forma genérica e flexível.

Descrição:

O usuário deve digitar números inteiros positivos.

A inserção encerra quando o usuário digitar um número negativo.

Cada número é armazenado em um nó de uma lista encadeada simples, composta

por estruturas do tipo:

*/

typedef struct Noh {

int valor;

struct Noh *prox;

} Noh;

/**

Após a inserção, o programa deve:

Exibir os valores originais;

Aplicar as seguintes transformações:

Nós de posição par (0, 2, 4, ...) → dobrar o valor (*valor *= 2)

Nós de posição ímpar (1, 3, 5, ...) → somar 10 (*valor += 10)

Exibir a lista após as transformações.

A lógica de aplicação das operações deve ser feita por meio de uma

função genérica com ponteiro para função, conforme o modelo:

*/

void aplicar_operacao(Noh *inicio, void (*operacao_par)(int *), void (*operacao_impar)(int *));

/**

Requisitos Técnicos:

Utilizar ponteiro para função para aplicar as operações (callback).

A função aplicar_operacao() deve percorrer a lista, identificar se a posição é par ou ímpar e chamar o callback correspondente.

Modularizar o código em três arquivos: main.c, funcoes.c, funcoes.h.

Utilizar malloc/free corretamente.

Garantir liberação completa da memória.

Comentar o código explicando as operações com ponteiros.

=====

Exemplo de execução esperada:

Digite números positivos (negativo para sair):

Número: 5

Número: 7

Número: 3

Número: 8

Número: -1

Lista original:

5 -> 7 -> 3 -> 8

Lista após aplicar operações:

10 -> 17 -> 6 -> 18

=====

Quesito	Nota obtida	
Estrutura da lista	0,0	
Inserção	1,0	
Exibição	0,25	
Callback	1,0	
Função de operação	0,25	
Função de operação	0,25	
Aplicação condicional	1,0	
Modularização	1,0	
Liberação de memória	0,25	
Total:	5,0	

*/